

Runner Right-Sizing Playbook

What is this playbook for?

This playbook helps you find the smallest runner size that can reliably run your GitHub Actions jobs within your time targets.

Many teams bump up to larger runners "to be safe" and never check if they could use something smaller. This playbook shows you how to test different runner sizes and pick the right one based on real data.

Who should use this playbook?

Use this playbook if:

- Your GitHub Actions costs are high or increasing
- You're not sure if your current runner size is the right choice
- You want to reduce costs without making your jobs slower
- You have workflows that run frequently (on every PR, nightly, etc.)

Step 0 — Choose the right job to test

Start with a job that makes sense to optimize. Choose a job that is:

- Expensive: Runs often (every PR, daily) OR uses large runners (16+ cores)
- Representative: Primary jobs in your repository
- Stable: Not failing randomly for unrelated reasons

Good candidates to test first

- Regression test suites that run on every pull request
- Large builds or big compilation jobs
- Integration or end-to-end test suites
- Any job that currently uses 16-core or 32-core runners

NOT good for your first test

- Small, quick jobs (lint checks, format checks) - they're already cheap
- Jobs that rarely run (once a month admin workflows)

Step 1 — Record your starting point (baseline)

Before you change anything, write down how things work right now.

For your chosen job, record these details:

1. Current runner label

What runner does this job use today?

Example: `ubuntu-linux-32core` or `ubuntu-linux-16core`

2. Typical runtime

How long does this job usually take?

- Go to your GitHub Actions page
- Look at the last 10-20 successful runs of this job
- Find the median (middle) runtime

Why median? Some runs might be unusually fast or slow. The median gives you a typical, normal runtime.

Example: "This job usually takes about 28-30 minutes"

3. Known problems

Does this job have any recurring issues?

- Out of memory (OOM) errors?
- "No space left on device" disk errors?
- Frequent timeouts?
- Occasional random failures?

Example: "Sometimes fails with disk space errors on smaller runners"

4. Your time target (Service Level Objective / SLO)

How fast does this job need to be for your team?

This depends on the type of workflow:

For PR checks (developer feedback):

- Developers wait for these results before continuing work
- Target example: "Should finish within 20-25 minutes"
- Faster is better because it keeps developers productive

For nightly builds or scheduled jobs:

- Nobody is actively waiting
- Target example: "60-90 minutes is fine if it saves money"
- Slower is acceptable if it's much cheaper

Finding your runtime data

You can find job runtimes in the [GitHub Actions UI](#). Each completed workflow run shows the duration of each job.

Important: You only need approximate numbers here. We're looking for big differences (like 10+ minutes), not precise seconds.

Step 2 — Run a multi-size experiment

Create a short-lived branch and run the same job on multiple runner sizes via a matrix.

Example (GitHub Actions; simplified):

```
jobs:
  regression-tests:
    strategy:
      matrix:
        runner:
          - ubuntu-linux-4core
          - ubuntu-linux-8core
          - ubuntu-linux-16core
          - ubuntu-linux-32core
    runs-on: ${matrix.runner}
    steps:
```

```
- uses: actions/checkout@v4
# ... your usual steps ...
```

Then:

- Trigger the workflow
- Let all matrix jobs complete
- Repeat 1–2 times if results look noisy

Record for each size:

- Runtime
- Success/failure
- Error type (OOM, disk full, timeouts)

Optional (if you have metrics): CPU %, memory, disk, I/O wait. If you don't, runtime + failures is still enough to decide.

Step 3 — Calculate cost per run

Now let's figure out how much each runner size costs.

How to calculate cost

For each runner size you tested:

1. Find the price per minute for that runner
 - Check your cloud provider's pricing page
 - Or check your company's internal billing dashboard
 - Or ask your platform/infrastructure team
2. Calculate: $\text{cost per run} = \text{runtime} \times \text{price per minute}$

Example calculation

Let's say you tested 4 different runner sizes. Here's what the data might look like:

Runner Size	Runtime (min)	Price per minute (\$)	Cost per run (\$)

4 cores	36	0.031	1.12
8 cores	31	0.047	1.46
16 cores	29	0.079	2.29
32 cores	28	0.143	4.00

Step 4 — Choose the best runner size

Find where more cores stop helping

Look at your runtime data:

- Does 4 → 8 cores give a big speedup? That's good, the job can use more cores.
- Does 8 → 16 cores barely improve? You've hit diminishing returns - more cores don't help much.
- Does 16 → 32 cores save only a minute or two? The job is maxed out.

Why this matters: If more cores don't speed things up, your job is limited by something else (memory, disk I/O, network). You're paying for cores you can't use.

Eliminate any runners that fail

If smaller runners fail with OOM, disk full, or timeouts - you can't use them, even if they're cheaper.

Pick the smallest runner that meets your SLO

For PR checks: Choose a runner that finishes with 10-20% buffer under your target.

For nightly/scheduled jobs: You can use smaller runners if there are no strict runtime requirements.

Decision criteria

Pick the smallest runner that:

- ☒ Doesn't cause failures
- ☒ Meets your SLO with 10-20% buffer
- ☒ Is past the point where more cores give meaningful gains

Example: From the Step 3 table, 8 cores costs \$1.46 and runs in 31 min. The 32-core runner only saves 3 minutes but costs \$4.00 (174% more). Choose 8 cores.

Step 5 — Roll out gradually and monitor

Rollout strategy

1. Update the `runs-on` label for one workflow (or non-main branches first)
2. Keep other workflows unchanged
3. Monitor for 1-2 weeks
4. If stable, apply to similar jobs and repeat for other workflows

What to watch

Runtime: Check your slowest runs (p95/p99 if available). A few minutes variance is normal, but if jobs regularly miss your target, you may need a larger runner.

Failures: Watch for new OOM, disk, or timeout errors. Investigate if they're related to the runner change.

Cost: Verify savings in your billing dashboard. Example: \$2.50 saved per run × 100 runs/week = \$1,000/month saved.

If something goes wrong

Revert by changing `runs-on`: back to your old runner, then investigate.

Quick reference checklist

Use this checklist when you run through the playbook:

Step 0: Choose your job

- ☐ Pick a heavy, frequently-run workflow job
- ☐ Make sure it's representative of real work (build, regression, integration tests)
- ☐ Confirm it's not randomly flaky

Step 1: Record your baseline

- ☐ Write down current runner label (e.g., "32-core Linux")
- ☐ Find median runtime from last 10-20 successful runs
- ☐ Note any known issues (OOM, disk, timeouts)
- ☐ Decide your runtime target/SLO (how fast does this need to be?)

Step 2: Run the experiment

- ☐ Create a test branch
- ☐ Add a matrix with 3-4 runner sizes (e.g., 4/8/16/32 cores)
- ☐ Trigger the workflow and let all jobs complete
- ☐ Repeat 1-2 times if results seem inconsistent
- ☐ Record for each size: runtime, success/failure, any error messages

Step 3: Calculate costs

- ☐ Find price per minute for each runner size
- ☐ Calculate cost per run = runtime × price per minute
- ☐ Make a simple table comparing runtime vs cost

Step 4: Choose the best runner

- ☐ Identify where adding more cores stops helping much
- ☐ Eliminate any sizes that cause failures
- ☐ Pick the smallest size that meets your SLO with 10-20% buffer
- ☐ Verify it has good cost/performance vs larger sizes

Step 5: Roll out and monitor

- ☐ Update the runner size in your workflow file
- ☐ Roll out to one workflow or non-main branches first
- ☐ Monitor for 1-2 weeks: runtime, failures, cost trends
- ☐ If stable: apply to similar jobs, repeat for other workflows
- ☐ Share before/after results with your team